Advanced Retrieval - Teaching Guide

What to show, what to say and which lines of code fix each problem. Folder: production_rag/advanced_retrieval

How every demo goes

1 Show the document	Open the source file in VS Code and point at the paragraph named in this guide.
2 Run, show the ISSUE	Start the script. It stops after each step (press Enter). Point at the red PROBLEM box.
3 Explain the FIX	Walk through the highlighted code lines in this guide, top to bottom.
4 Show the PROOF	The green FIX box: same question, better chunk. Trace the data with the 'How the data changes' table.
5 Show it in Qdrant	At the end, script 05 stores every chunk in Qdrant. Show the same data as database points.

Setup (once, before class)

Terminal

```
cd "$HOME/Documents/Agentic_AI_P2/Advanced Rag/production_rag"
source .venv/bin/activate
pip install -r advanced_retrieval/requirements.txt
docker start qdrant # the Qdrant container from the main project
cd advanced_retrieval
python -c "from rag_core import embed; print(embed(['hello']).shape)" # downloads model, prints (1, 384)
```

Script 04 reads the Gemini key from **production_rag/.env**. With no key or no quota it prints **OFFLINE: replaying a saved LLM answer** and still works. Script 05 reads **QDRANT_URL** from the same file. If Docker is not running it uses a local folder instead (no dashboard).

Run order

Script	Concept	Issue students see	Time
01_table_aware_chunking.py	Table-aware + sentence chunking	A table row loses its column headers	15 min
02_metadata_filtering.py	Metadata filtering	Old (2024) and US policies come back for an India user	15 min
03_hybrid_retrieval.py	Hybrid search (BM25 + vector + RRF)	Keyword misses paraphrases; vector confuses error codes	20 min
04_query_rewriting.py	Query rewriting	Follow-ups, typos and vague words find the wrong chunks	15 min
05_store_in_qdrant.py	Everything in the database	None: shows points, payload and a filtered search in Qdrant	10 min

Files you will open

01-05 *.py	The demos. The important logic is written inside each script so you can explain it line by line.
rag_core.py	Shared pieces: load documents, sentence chunking, embedding, BM25, RRF.
display.py	Only prints coloured tables. Tell students to ignore it.
data/	The documents (next page) and llm_cache.json (saved Gemini answers for offline mode).
output/	Script 01 writes chunks_table_aware.json here. Script 05 reads it.

Before class: run all five once with **--no-pause**. Script 01 output is always the same. Scripts 02-05 depend on the embedding model, so check that the vector rankings still tell the story in this guide.

The documents

Seven small documents. Each one is there to cause one specific retrieval problem. Show them to students before running anything: if they have seen the source, they can follow every transformation.

File (in data/)	What is inside	The trap it sets	Used in
employee_handbook.pdf	2 headed sections, Table 1 (leave days for grades L1-L5), Table 2 (laptop per role)	A table flattened into numbers loses its headers	01, 05
docs/hr/leave_policy_india_2025.txt	India 2025: 24 days paid leave, carry forward 10	The CORRECT answer for an India employee	02-05
docs/hr/leave_policy_india_2024.txt	India 2024 (old): 18 days, no carry forward	Old version, almost the same sentence	02, 05
docs/hr/leave_policy_us_2025.txt	US: 15 days PTO, roll over 5	Other country, almost the same sentence	02, 04, 05
docs/hr/remote_work_policy_2025.txt	'perform their duties remotely', 'Telecommuting staff'	Formal words users never type	03
docs/it/vpn_error_codes.txt	ERR-4012, ERR-4013, ERR-4021, ERR-5001 + fixes	Codes that look alike to an embedding model	03, 04
docs/finance/travel_reimbursement_2025.txt	Taxi to airport reimbursed, how to claim, meal limits	'cab' vs 'taxi', 'expense my' vs 'reimbursed'	04

How a .txt document is built

Every .txt file has two parts. Between the two --- lines is the **metadata header**: facts ABOUT the document. Below it is the **body**: paragraphs separated by a blank line. The code turns each paragraph into one or more chunks and copies the header onto every chunk.

data/docs/hr/leave_policy_india_2025.txt

```
---
title: Leave Policy (India)
department: HR
region: India
year: 2025
doc_type: policy
---
```

Full-time employees in India receive 24 days of paid annual leave per calendar year. Leave is credited at the start of each quarter, six days at a time.

Up to 10 unused days can be carried forward to the next year. Employees must apply for leave in the HR portal at least one week in advance.

Grey = metadata header. Each paragraph below it becomes a chunk.

How the PDF looks (open it in Preview)

Acme Corp - Employee Handbook 2025. Section 1 'Leave Policy': a paragraph, then **Table 1: Leave entitlement by grade**, then a paragraph about sick leave. Section 2 'Laptop Allowance': a paragraph, then **Table 2: Laptop allowance by role**, then a paragraph about returning laptops.

Grade	Annual leave (days)	Sick leave (days)	Carry forward (days)
L1	18	8	5
L2	20	10	5
L3	24	12	10
L4	26	12	10
L5	30	15	15

Table 1 as it appears in the PDF. The L3 row is the one we follow through script 01 and script 05.

One chunk, end to end (the big picture)

How the data changes, step by step

1. Document leave_policy_india_2025.txt	`Full-time employees in India receive 24 days of paid annual leave per calendar year. Leave is credited at the start of each quarter, six days at a time.`
2. Metadata parse_document()	`{'title': 'Leave Policy (India)', 'department': 'HR', 'region': 'India', 'year': 2025, 'doc_type': 'policy', 'source': 'leave_policy_india_2025.txt'}`
3. Chunk text + metadata	`{'id': 'leave_policy_india_2025#5', 'text': 'Full-time employees in India receive 24 ...', 'metadata': {...the dict above...}}`
4. Vector embed()	`[0.012, -0.087, 0.041, ... ]  384 numbers that capture the MEANING of the text`
5. Qdrant point script 05	`id: 5  vector: 384 numbers  payload: {text, title, department, region: 'India', year: 2025, source, chunk_type: 'text'}`

Vector numbers above are illustrative; the script prints the real first numbers.

01 Table-aware chunking

Question: **How many sick leave days does an L3 employee get?** Correct answer: **12**, from Table 1 of the handbook.

Why this problem happens

A PDF has no idea what a table is. It stores letters at x/y positions on the page. When a text extractor reads the page, it reads left to right, top to bottom, so a table comes out as plain lines: first the header line, then one line of numbers per row.

Fixed-size chunking then cuts this text every N characters, wherever that falls. The header line ends up in one chunk and the row "L3 24 12 10" in another. The retriever can find the row, but the chunk only contains four numbers. Which one is sick leave? The LLM has to guess, and it will guess confidently.

The fix is to treat tables as tables: find them first, write every row so it carries its own column names, and only then chunk the normal text, by whole sentences so no sentence is cut in half.

`data/employee_handbook.pdf (page 1, Section 1)`

1. Leave Policy - This handbook explains leave, equipment and travel rules for Acme Corp employees in India. ... Leave entitlement depends on your job grade. The table below lists the number of days for each grade.

Table 1: Leave entitlement by grade [the table shown on the documents page, L3 row: 24 | 12 | 10]

Sick leave of more than two consecutive days requires a medical certificate. Unused sick leave cannot be carried forward. All leave must be applied for in the HR portal.

Open the PDF and point at Table 1, row L3. Ask: which column answers our question? (Sick leave)

How the data changes, step by step

1. In the PDF employee_handbook.pdf	`A real table: header row + 5 rows. Row L3 = 24 | 12 | 10`

2. Extracted text
Step 1, line 36
	`Grade Annual leave (days) Sick leave (days) Carry forward (days)`
	`L1 18 8 5`
	`L2 20 10 5`
	`L3 24 12 10`
	`...`

3. Naive chunk 2 Step 1, line 42	`' 5\nL3 24 12 10\nL4 26 12 10\nL5 30 15 15\nSick leave of more than two ...'` `-> no headers. The header line is in chunk 1.`

4. Table detected
Step 2, lines 63-69
	`caption: 'Table 1: Leave entitlement by grade'`
	`rows[0] = ['Grade', 'Annual leave (days)', 'Sick leave (days)', 'Carry forward (days)']`
	`rows[3] = ['L3', '24', '12', '10']`

5. Row chunk Step 3, lines 101-104	`Table 1: Leave entitlement by grade | Grade: L3 | Annual leave (days): 24 | Sick leave (days): 12 | Carry forward (days): 10`
6. Qdrant point script 05, step 4	`id: 17  payload: {text: 'Table 1: ... Grade: L3 ...', source: 'employee_handbook.pdf',` `chunk_type: 'table_row', page: 1, region: 'India', year: 2025, ...}`

Run

```
python 01_table_aware_chunking.py
```

ISSUE - Step 1 on screen: naive chunking (cut every 200 characters)

```
2 | ' 5\nL3 24 12 10\nL4 26 12 10\nL5 30 15 15\nSick leave of more than two
  | consecutive days requires a medical certificate. ...'
```

Point at **L3 24 12 10** in chunk 2, then at the header line in chunk 1. Say: *"Which of these four numbers is sick leave? The LLM can't know either."*

01_table_aware_chunking.py lines 35-42

```
35 with pdfplumber.open(PDF_PATH) as pdf:
36     full_text = "\n".join(page.extract_text() for page in pdf.pages)
37
38 console.print("[dim]Raw text that pdfplumber gives us (tables become plain lines):[/dim]")
39 console.print(full_text)
40
41 CHUNK_SIZE = 200
42 naive_chunks = [full_text[i : i + CHUNK_SIZE] for i in range(0, len(full_text), CHUNK_SIZE)]
```

Line	What it does - say this
36	extract_text() flattens the whole page. The table becomes plain lines of numbers.
42	Fixed-size chunking: slice every 200 characters, wherever that lands. This is the problem.

FIX, part 1 - find every table (Step 2)

01_table_aware_chunking.py lines 61-70

```
61 with pdfplumber.open(PDF_PATH) as pdf:
62     for page_number, page in enumerate(pdf.pages, start=1):
63         for t in page.find_tables():          # detects tables from their grid lines
64             x0, top, x1, bottom = t.bbox
65             # The caption ("Table 1: ...") is the line of text just ABOVE the table.
66             caption_area = page.crop((0, max(0, top - 22), page.width, top))
67             caption = (caption_area.extract_text() or "").strip()
68             found_tables.append(
69                 {"page": page_number, "bbox": t.bbox, "caption": caption, "rows": t.extract()}
70             )
```

Line	What it does - say this
63	find_tables() detects tables from their grid lines. One object per table on the page.
64	bbox = the table's rectangle on the page (left, top, right, bottom). We need it in Step 4.
66-67	Crop a 22-point strip just above the table and read it: that is the caption "Table 1: ...".
69	t.extract() returns the table as a list of rows. Row 0 = the header, the rest = data.

FIX, part 2 - turn each table into chunks that stand alone (Step 3)

01_table_aware_chunking.py lines 88-107

```
for t in found_tables:
    header, data_rows = t["rows"][0], t["rows"][1:]

    if len(data_rows) <= SMALL_TABLE_ROWS:
        # Whole table as markdown - an LLM reads markdown tables very well.
        lines = [t["caption"], "| " + " | ".join(header) + " |", "|" + "---|" * len(header)]
        lines += ["| " + " | ".join(row) + " |" for row in data_rows]
        table_chunks.append(
            {"text": "\n".join(lines), "metadata": {"type": "table", "page": t["page"], "caption":
t["caption"]}}
        )
    else:
        # One self-describing chunk per row: "Grade: L3 | Sick leave (days): 12 | ..."
        for row in data_rows:
            pairs = [f"{column}: {value}" for column, value in zip(header, row)]
            table_chunks.append(
                {
                    "text": f"{t['caption']} | " + " | ".join(pairs),
                    "metadata": {"type": "table_row", "page": t["page"], "caption": t["caption"]},
                }
            )
```

Line	What it does - say this
89	Split the rows into header and data_rows.
91-94	**Small table** (4 rows or fewer, like Table 2): keep it whole as ONE markdown chunk. LLMs read markdown tables well.
101	**Big table** (Table 1 has 5 rows): zip() pairs every column name with the value in the same position -> "Sick leave (days): 12".
104	One chunk per row, starting with the caption. Every row chunk now carries its own headers.

FIX, part 3 - remove the tables, then chunk the text by sentences (Step 4)

01_table_aware_chunking.py lines 121-134

```
with pdfplumber.open(PDF_PATH) as pdf:
    for page_number, page in enumerate(pdf.pages, start=1):
        text_only = page
        for t in [t for t in found_tables if t["page"] == page_number]:
            x0, top, x1, bottom = t["bbox"]
            # cut out the table AND its caption so they are not indexed twice
            text_only = text_only.outside_bbox((x0, max(0, top - 22), x1, bottom))
        page_text = text_only.extract_text()

        console.print(f"[dim]Page {page_number} text with tables removed:[/dim]")
        console.print(page_text)

        for chunk in sentence_chunks(page_text, max_chars=300, overlap=1):
            text_chunks.append({"text": chunk, "metadata": {"type": "text", "page": page_number}})
```

Line	What it does - say this
127	outside_bbox() keeps everything EXCEPT the table and its caption, so the table is not indexed twice.
133	Chunk the leftover text with sentence_chunks(): max 300 characters, 1 sentence of overlap (code below).

rag_core.py lines 60-79

```
60  def sentence_chunks(text, max_chars=350, overlap=1):
61      """Group whole sentences into chunks of at most ~max_chars characters.
62
63      - Never cuts a sentence in half (unlike fixed-size chunking).
64      - `overlap` = how many sentences from the end of one chunk are repeated
65        at the start of the next one, so context is not lost at the border.
66      """
67      sentences = split_sentences(text)
68      chunks, current = [], []
69
70      for sentence in sentences:
71          candidate = " ".join(current + [sentence])
72          if current and len(candidate) > max_chars:
73              chunks.append(" ".join(current))           # close the current chunk
74              current = current[-overlap:] if overlap else []
75          current.append(sentence)
76
77      if current:
78          chunks.append(" ".join(current))
79      return chunks
```

Line	What it does - say this
67	Split the text into sentences first.
71-73	Try adding the next sentence. If the chunk would pass max_chars, close the chunk. A sentence is never cut.
74	**Overlap**: start the next chunk with the last sentence of the previous one, so a fact on the border is not lost.
75	Add the sentence to the current chunk.

On screen: text chunk 0 ends with "The table below lists the number of days for each grade." and chunk 1 starts with the same sentence. That repeated sentence is the overlap.

PROOF - Step 5: same search over both chunk sets (lines 150-151)

```
BEFORE (naive)       ' 5\nL3 24 12 10\nL4 26 12 10\nL5 30 15 15\nSick leave of more ...'
AFTER (table-aware)  Table 1: Leave entitlement by grade | Grade: L3 | Annual leave (days): 24 |
                     Sick leave (days): 12 | Carry forward (days): 10
```

"Saved 10 chunks to output/chunks_table_aware.json": 5 row chunks, 1 whole-table chunk (Table 2), 4 text chunks. Open the JSON file to show the final chunks. Script 05 stores exactly these in Qdrant.

Ask: Why do we cut the table out of the page before sentence chunking? *(So the same numbers don't end up in two chunks, once with headers and once without.)*

02 Metadata filtering

User: an **employee in India, 2025**. Question: **How many paid leave days do I get?** Correct answer: **24**.

Why this problem happens

An embedding measures what a text is ABOUT, not whether it is valid for this user. The three leave policies below are the same sentence with a different number and a different country. Their vectors are almost identical, so a pure similarity search ranks them almost equally. Whichever wins, the LLM answers with that number: 24, 18 or 15.

No prompt can fix this, because the wrong chunk is already in the context. The fix has to happen BEFORE ranking: store facts about each chunk (region, year, department, source) next to its vector, and use them as hard rules. Similarity then only ranks chunks that are allowed to be the answer.

Where do the filter values come from? From the logged-in user (their country), the app (the current year), or a dropdown. Not from the LLM: access and validity rules must be decided by code.

Show the three leave policies side by side

data/docs/hr/leave_policy_india_2025.txt

```
---
title: Leave Policy (India)
department: HR
region: India
year: 2025
doc_type: policy
---
```

Full-time employees in India receive 24 days of paid annual leave per calendar year. Leave is credited at the start of each quarter, six days at a time.

data/docs/hr/leave_policy_india_2024.txt

```
---
title: Leave Policy (India) - OLD VERSION
department: HR
region: India
year: 2024
doc_type: policy
---
```

Full-time employees in India receive 18 days of paid annual leave per calendar year. Leave is credited monthly at 1.5 days per month.

data/docs/hr/leave_policy_us_2025.txt

```
---
title: Leave Policy (US)
department: HR
region: US
year: 2025
doc_type: policy
---
```

Full-time employees in the United States (US) receive 15 days of paid annual leave, called paid time off (PTO), per year. PTO accrues every pay period.

Say: "Read the three yellow sentences. Only the number and the country differ. The difference is in the header, and the header is exactly what basic RAG throws away."

How the data changes, step by step

1. File header between the --- lines	`region: India` `year: 2025`
2. Python dict Step 2, lines 41-46	`{'title': 'Leave Policy (India)', 'department': 'HR', 'region': 'India', 'year': 2025,` `'doc_type': 'policy', 'source': 'leave_policy_india_2025.txt'}`
3. Chunk BEFORE Step 3	`"Full-time employees in India receive 24 days of paid annual leave per calendar year. ..."`
4. Chunk AFTER Step 3, line 65 / 78	`{'id': 'leave_policy_india_2025#0', 'text': 'Full-time employees in India ...',` `'metadata': {'region': 'India', 'year': 2025, 'department': 'HR', ...}}`
5. Filter check Step 6, matches()	`where = {'region': {'$in': ['India', 'Global']}, 'year': 2025}` `India 2025 -> True  India 2024 -> False (year)  US -> False (region)`
6. Qdrant point script 05, step 4	`payload: {text, region: 'India', year: 2025, ...}` `filter: FieldCondition(key='region', MatchAny(['India','Global'])) AND year == 2025`

> **Run**
>
> **`python 02_metadata_filtering.py`**

Steps 1-3: show what metadata is and how it is attached

`02_metadata_filtering.py` lines 36-47

```
36  def parse_document(path):
37      raw = Path(path).read_text(encoding="utf-8")
38      _, header, body = raw.split("---", 2)        # ['', header, body]
39
40      metadata = {}
41      for line in header.strip().splitlines():     # "region: India"
42          key, value = line.split(":", 1)          # -> "region", " India"
43          metadata[key.strip()] = value.strip()
44
45      metadata["year"] = int(metadata["year"])     # store numbers as numbers
46      metadata["source"] = Path(path).name         # remember where it came from
47      return metadata, body.strip()
```

Line	What it does - say this
38	split("---", 2) cuts the file at the --- lines: ["", header, body].
41-43	Read each "key: value" line into a dict. This dict IS the metadata.
45	year becomes an int, so comparisons like >= 2025 work later.
46	source = file name. Every answer can be traced back to its file.

`02_metadata_filtering.py` lines 73-78

```
73  chunks = []
74  for path in sorted(DOCS_DIR.rglob("*.txt")):
75      metadata, body = parse_document(path)
76      for paragraph in body.split("\n\n"):
77          for text in sentence_chunks(paragraph, max_chars=200, overlap=0):
78              chunks.append({"id": f"{path.stem}#{len(chunks)}", "text": text, "metadata": dict(metadata)})
```

Line	What it does - say this
78	**Attach**: every chunk becomes {id, text, metadata}. dict(metadata) gives each chunk its OWN copy of the file's metadata. On screen, Step 3 prints the chunk BEFORE (just a string) and AFTER (with metadata).

> **ISSUE - Step 5 on screen: search all chunks, no filter**
>
> Look at the last column, **allowed for this user?**. Rows marked **OUTDATED (2024)** or **WRONG COUNTRY (US)** sit in the top results with scores close to the correct chunk. Say: *"Similarity measures topic, not validity."*
>
> If no wrong row appears on your machine, the script says so. Make the point anyway: nothing PREVENTS it.

FIX - filter first, then rank (Step 6)

02_metadata_filtering.py lines 111-115

```
111 def search(query, candidate_indexes, top_k=4):
112     """Rank only the candidate chunks by cosine similarity."""
113     query_vector = embed([query])[0]
114     scored = [(i, float(vectors[i] @ query_vector)) for i in candidate_indexes]
115     return sorted(scored, key=lambda item: -item[1])[:top_k]
```

Line	What it does - say this
114	search() only scores the chunks in candidate_indexes. The filter decides that list.

02_metadata_filtering.py lines 147-173

```
147     for key, condition in where.items():
148         value = metadata.get(key)
149         if isinstance(condition, dict):                    # operator form
150             for operator, target in condition.items():
151                 if operator == "$eq" and not value == target:
152                     return False
153                 if operator == "$ne" and not value != target:
154                     return False
155                 if operator == "$gte" and not value >= target:
156                     return False
157                 if operator == "$lte" and not value <= target:
158                     return False
159                 if operator == "$in" and value not in target:
160                     return False
161         elif value != condition:                           # plain equality
162             return False
163     return True
164
165
166 # The filter comes from the LOGGED-IN USER's profile - not from the question text.
167 where = {"region": {"$in": ["India", "Global"]}, "year": 2025}
168 console.print(f"where = {where}")
169
170 candidates = [i for i, c in enumerate(chunks) if matches(c["metadata"], where)]
171 console.print(f"Filter kept [bold]{len(candidates)}[/bold] of {len(chunks)} chunks -> only these are
    ranked.")
172
173 after = search(QUERY, candidate_indexes=candidates)
```

Line	What it does - say this
147-148	matches() checks every condition in the where dict against one chunk's metadata.
149-159	Operator form, the same style vector databases use: $eq, $ne, $gte, $lte, $in. $in = the value must be one of a list.
161	Plain form: {"year": 2025} means equals.
167	**The filter comes from the logged-in user's profile**, not from the question: region India or Global, year 2025.
170	Pre-filter: keep only the chunks that match. On screen: "Filter kept 11 of 15 chunks".
173	Rank only those. Wrong-country and old chunks can no longer be returned.

PROOF - Step 6 table

Every row in **With metadata filter** says **ok**. The answer will be 24 days. Step 7 repeats the idea with {"department": "IT"}. Script 05 does the same filter inside Qdrant.

Ask: Who should decide the filter, the user's profile or the LLM? Why? *(The profile. An LLM can suggest a year, but access and region rules must never come from it.)*

03 Hybrid retrieval: BM25 + vector + RRF

Two search engines over the same 15 chunks, merged into one ranking.

Why this problem happens

Keyword search (BM25) compares words. It is precise: "err-4012" matches only the chunk that contains "err-4012". But it has no idea what words mean. A user who writes "work from my house" shares no word with "perform their duties remotely", so BM25 finds nothing.

Vector search compares meaning. It understands that "house" and "remotely" are related. But meaning is fuzzy: ERR-4012, ERR-4013 and ERR-4021 are all "a VPN error code and its fix", so their vectors are close together and the wrong code can win.

Each one fails exactly where the other one is strong. So we run both and merge their result lists. We cannot add their scores (BM25 gives about 0-10, cosine gives 0-1), so we merge by RANK with Reciprocal Rank Fusion: a chunk near the top of both lists wins.

data/docs/it/vpn_error_codes.txt

```
---
title: VPN Error Codes
department: IT
region: Global
year: 2025
doc_type: troubleshooting
---
```

ERR-4012: The VPN certificate on your laptop has expired. Open the IT Self-Service app, click Renew Certificate, then restart the VPN client.

ERR-4013: Your VPN password has expired. Reset it on the password portal and wait 15 minutes before reconnecting.

ERR-4021: The VPN server is unreachable. Check your internet connection and try the backup server vpn2.acme.example.

ERR-5001: Multi-factor authentication timed out. Approve the push notification on your phone within 60 seconds.

Case A uses this file. Point out that the paragraphs have the same shape: code, problem, fix. Only the code is unique.

data/docs/hr/remote_work_policy_2025.txt

```
---
title: Remote Work Policy
department: HR
region: Global
year: 2025
doc_type: policy
---
```

Employees may perform their duties remotely for up to two days in each week, subject to manager approval. Remote days must be recorded in the attendance system.

Telecommuting staff must connect through the company VPN when accessing internal systems.

Case B uses this file. Ask: which words would a normal person type? (work from home, house, job) None of them appear here.

How the data changes, step by step

1. Paragraph vpn_error_codes.txt	`ERR-4012: The VPN certificate on your laptop has expired. Open the IT Self-Service app, ...`
2. Chunk load_knowledge_base()	`id: vpn_error_codes#11  one paragraph = one chunk (short paragraphs fit in 200 characters)`
3. Keyword view tokenize(), line 35	`['err-4012', 'vpn', 'certificate', 'laptop', 'has', 'expired', 'open', ' self-service',` ` 'app', 'click', 'renew', 'certificate', 'then', 'restart', 'vpn', 'client']`
4. Meaning view embed(), line 85	`384 numbers. Very close to the vectors of the ERR-4013 and ERR-4021 chunks.`
5. Query Case A	`"How do I fix ERR-4012?"  ->  tokens ['fix', 'err-4012']  +  a 384-number query vector`
6. Two rankings lines 126-127	`BM25: only chunk #11 contains 'err-4012'  ->  rank 1` `Vector: #11, #12, #13 score almost the same`
7. Fused rrf(), line 138	`chunk #11 is in both lists  ->  1/(60+1) + 1/(60+r) beats a chunk found by one list only`

Run

```
python 03_hybrid_retrieval.py
```

ISSUE - the two failure cases (Steps 5 and 6 on screen)

Case A, "How do I fix ERR-4012?" - Vector table: ERR-4012, ERR-4013 and ERR-4021 have very close cosine scores, so the wrong code can rank first. BM25 finds exactly one chunk.

Case B, "Can I do my job from my house sometimes?" - BM25 table says **NO keyword match at all**: the tokens are ['job', 'house', 'sometimes'] and none is in any document. Vector search finds the remote-work policy.

Say: "Keyword search knows words, not meaning. Vector search knows meaning, not exact codes."

Keyword search - BM25 (Steps 1-2)

03_hybrid_retrieval.py lines 34-36

```
34  def tokenize(text):
35      words = re.findall(r"[a-z0-9]+(?:-[a-z0-9]+)*", text.lower())   # keeps "err-4012" as ONE word
36      return [w for w in words if w not in STOPWORDS]
```

Line	What it does - say this
35	Lowercase, split into words. The (?:-[a-z0-9]+)* part keeps "err-4012" as ONE word. Line 36 drops stopwords like "how" and "the".

03_hybrid_retrieval.py lines 53-73

```
53  doc_tokens = [tokenize(t) for t in texts]
54  N = len(doc_tokens)
55  avg_len = sum(len(d) for d in doc_tokens) / N
56  doc_freq = Counter(word for tokens in doc_tokens for word in set(tokens))
57  K1, B = 1.5, 0.75
58
59
60  def bm25_search(query, top_k=5):
61      scores = []
62      for tokens in doc_tokens:
63          counts = Counter(tokens)
64          score = 0.0
65          for word in tokenize(query):
66              if word not in counts:
67                  continue
68              idf = math.log(1 + (N - doc_freq[word] + 0.5) / (doc_freq[word] + 0.5))
69              tf = counts[word]
70              score += idf * tf * (K1 + 1) / (tf + K1 * (1 - B + B * len(tokens) / avg_len))
71          scores.append(score)
72      ranked = sorted(range(N), key=lambda i: -scores[i])
73      return [(i, scores[i]) for i in ranked if scores[i] > 0][:top_k]   # only real matches
```

Line	What it does - say this
56	doc_freq = in how many chunks each word appears.
66	A chunk only scores for query words it actually contains.
68	**IDF**: rare word -> big weight. On screen: err-4012 is in 1 chunk (IDF 2.37), employees is in 5 chunks (IDF 1.07). The rare code counts more than twice as much.
70	**TF part**: more repeats score higher but level off (K1). B gives long chunks a small penalty.
73	Return only chunks with score > 0. That is why Case B prints an empty table.

Vector search (Step 3)

03_hybrid_retrieval.py lines 90-94

```
90  def vector_search(query, top_k=5):
91      query_vector = embed([query])[0]
92      similarities = chunk_vectors @ query_vector           # cosine (vectors are normalised)
93      best = np.argsort(-similarities)[:top_k]
94      return [(int(i), float(similarities[i])) for i in best]
```

Line	What it does - say this
91	Turn the query into a 384-number vector (the chunks were embedded once, line 85).
92	Matrix x vector = cosine similarity of the query with every chunk at once (the vectors have length 1).

FIX - Reciprocal Rank Fusion (Step 4)

03_hybrid_retrieval.py lines 111-116

```
111 def rrf(ranked_lists, k=60):
112     fused = defaultdict(float)
113     for ranked in ranked_lists:
114         for rank, chunk_index in enumerate(ranked, start=1):
115             fused[chunk_index] += 1 / (k + rank)
116     return sorted(fused.items(), key=lambda item: -item[1])
```

Line	What it does - say this
111	Input: several ranked lists of chunk ids (here: the BM25 list and the vector list).
114-115	For every list a chunk appears in, add 1 / (60 + rank). Ranks, not scores, because the two scales differ.
116	Sort by the total. A chunk found by BOTH lists collects two parts and usually wins.

03_hybrid_retrieval.py lines 136-138

```
136     keyword_rank = {i: r for r, (i, _) in enumerate(keyword, start=1)}
137     vector_rank = {i: r for r, (i, _) in enumerate(semantic, start=1)}
138     fused = rrf([[i for i, _ in keyword], [i for i, _ in semantic]])
```

Line	What it does - say this
138	Hybrid = rrf() over the keyword ranking and the vector ranking.

PROOF - Step 7, "ERR-4012 popped up, how do I get my VPN working again?"

The HYBRID table prints how each score is built, for example `1/(60+1) + 1/(60+2) = 0.0325`. The ERR-4012 chunk ranks high in both lists, so it ends on top.

Do one sum on the board: rank 1 in BM25 and rank 2 in vector = 1/61 + 1/62 = 0.0325.

Ask: Which retriever wins for an error code? Which one for a paraphrase?

04 Query rewriting

Before searching, an LLM rewrites the user's message into a good search query. The script searches with the hybrid retriever from script 03.

Why this problem happens

Retrieval can only match what the user typed. Real messages are not search queries. A follow-up such as "what about the US?" has no topic: the topic (leave) is in the previous message. Typed in a hurry, "err4012" is a different token from the documented "ERR-4012", so keyword search misses it. And users use everyday words ("cab", "expense my") while policies use formal ones ("taxi", "reimbursed").

The retriever is not broken; its input is. So we fix the input. One LLM call turns the message into a clear, complete query in the documents' vocabulary, then we search with that. For vague questions we ask for several versions and merge their results with RRF.

The prompt always says "do not answer the question". The LLM only rewrites; the answer still comes from retrieved documents.

data/docs/hr/leave_policy_us_2025.txt

```
---
title: Leave Policy (US)
department: HR
region: US
year: 2025
doc_type: policy
---
```

Full-time employees in the United States (US) receive 15 days of paid annual leave, called paid time off (PTO), per year. PTO accrues every pay period.

Up to 5 unused PTO days roll over into the next year.

Case A target. The follow-up "what about the US?" should find this paragraph.

data/docs/finance/travel_reimbursement_2025.txt

```
---
title: Travel Reimbursement Policy
department: Finance
region: Global
year: 2025
doc_type: policy
---
```

Taxi and ride-hailing fares between home and the airport are reimbursed for approved business trips.

To get reimbursed, submit a travel expense claim in the expense portal within 30 days and attach all receipts.

Meal expenses on business trips are reimbursed up to 1,500 INR per day in India and 50 USD per day in the US.

Case C target. The user will ask about a 'cab'. The document never says 'cab'. Case B uses the ERR-4012 paragraph from the previous section.

How the data changes, step by step

1. User types Case A	`"what about the US?"  (after a question about India leave days)`
2. Prompt lines 100-109	`conversation history + follow-up + rules: standalone question, keep codes, do not answer,` `return JSON {"rewritten_query": ...}`
3. Rewritten line 113	`"How many paid leave days do employees in the US get?"`
4. Found line 116	`leave_policy_us_2025.txt: 'Full-time employees in the United States (US) receive 15 days ...'`
5. User types Case B	`"vpn not wrking!!! says err4012 pls help asap"  ->  tokens include 'wrking', 'err4012'`
6. Rewritten line 137	`"VPN not working error ERR-4012"  ->  tokens include 'err-4012' (exact match)`
7. User types Case C	`"Can I expense my cab to the airport?"`

8. 3 variants
line 159
	`'taxi fare reimbursement for airport trips'`
	`'how to submit a travel expense claim with receipts'`
	`'business trip transport costs policy'`

9. Fused line 167	`#1 Taxi and ride-hailing fares ... reimbursed  #2 To get reimbursed, submit a travel expense claim ...`

Rewrites above are the saved offline answers (data/llm_cache.json). A live Gemini call may word them slightly differently.

Run

`python 04_query_rewriting.py`

Top of the output: "Gemini key found: True/False". Each rewrite prints LIVE answer or OFFLINE.

The only LLM call

`04_query_rewriting.py` lines 47-66

```
47      if API_KEY:
48          try:
49              from google import genai
50
51              client = genai.Client(api_key=API_KEY)
52              response = client.models.generate_content(
53                  model=MODEL,
54                  contents=prompt,
55                  config={"response_mime_type": "application/json", "temperature": 0},
56              )
57              answer = json.loads(response.text)
58              console.print(f"[green]LIVE answer from {MODEL}[/green]")
59              cache[cache_key] = answer                    # save it for offline use next time
60              CACHE_PATH.write_text(json.dumps(cache, indent=2))
61              return answer
62          except Exception as error:                        # quota, network, bad model name...
63              console.print(f"[red]Gemini call failed:[/red] {short(str(error), 150)}")
64
65      console.print("[yellow]OFFLINE: replaying a saved LLM answer from data/llm_cache.json[/yellow]")
66      return cache[cache_key]
```

Line	What it does - say this
52-55	Call Gemini. response_mime_type = JSON forces a JSON reply; temperature 0 = the same rewrite every time.
57	Parse the JSON. Every prompt asks for {"rewritten_query": ...} or {"queries": [...]}.
59-60	Save the answer to data/llm_cache.json for next time.
65-66	No key, no quota or no internet: replay the saved answer. The class demo never breaks.

ISSUE - Case A (Step 1): a follow-up question

The BEFORE table is mostly unrelated chunks. Say: *"This sentence has no topic. The retriever doesn't know we are talking about leave."*

04_query_rewriting.py lines 99-116

```
 99  conversation = "\n".join(f"{m['role']}: {m['content']}" for m in history)
100  prompt_a = f"""You rewrite questions for a company knowledge-base search engine.
101  Use the conversation history to turn the follow-up into a complete, standalone question.
102  Keep names, codes and numbers exactly. Do not answer the question.
103
104  Conversation:
105  {conversation}
106
107  Follow-up question: {follow_up}
108
109  Return JSON: {{"rewritten_query": "..."}}"""
110
111  console.print("\n[bold]Prompt sent to the LLM:[/bold]")
112  console.print(prompt_a, style="dim")
113  rewritten = ask_llm("followup", prompt_a)["rewritten_query"]
114  console.print(f"\nRewritten query: [bold green]{rewritten}[/bold green]")
115
116  show(f"AFTER - search with {rewritten!r}", retriever.search(rewritten))
```

Line	What it does - say this
99	Turn the chat history into plain text for the prompt.
101-102	The rules: make a standalone question, keep codes exactly, **do not answer**.
113	Rewritten: "How many paid leave days do employees in the US get?"
116	Search again with the rewrite. The US leave policy comes to the top.

ISSUE - Case B (Step 2): messy input

Point at the keyword tokens: "err4012" is not "err-4012", so the BM25 line prints **[]** (no match for the code).

04_query_rewriting.py lines 130-141

```
130  prompt_b = f"""You rewrite questions for a company IT knowledge-base search engine.
131  Fix spelling, remove filler words, and write error codes in the official format ERR-NNNN.
132  Do not answer the question.
133
134  Question: {messy}
135
136  Return JSON: {{"rewritten_query": "..."}}"""
137  rewritten_b = ask_llm("messy", prompt_b)["rewritten_query"]
138  console.print(f"Rewritten query: [bold green]{rewritten_b}[/bold green]")
139  console.print(f"keyword tokens: {tokenize(rewritten_b)}")
140
141  show(f"AFTER - search with {rewritten_b!r}", retriever.search(rewritten_b))
```

Line	What it does - say this
131	Rule in the prompt: fix spelling and write codes as ERR-NNNN.
139	Tokens after the rewrite now contain err-4012, an exact keyword match.
141	Both retrievers now agree, so the ERR-4012 fix comes first.

ISSUE - Case C (Step 3): the user's words are not the document's words

Can I expense my cab to the airport? Show the travel document again: it says "taxi", "reimbursed", "expense claim".

04_query_rewriting.py lines 159-168

```
159 variants = ask_llm("multi", prompt_c)["queries"]
160
161 ranked_lists = []
162 for v in [question] + variants:
163     results = retriever.search(v, top_k=5)
164     ranked_lists.append([i for i, _ in results])
165     console.print(f"  query: [cyan]{v}[/cyan]  ->  top: {short(chunks[results[0][0]]['text'], 60)}")
166
167 fused = reciprocal_rank_fusion(ranked_lists)[:3]
168 show("AFTER - all variants fused with RRF", fused)
```

Line	What it does - say this
159	Ask Gemini for 3 differently-worded queries (multi-query).
162-164	Search with the original question AND each variant. Keep each ranked list.
167	Merge all the lists with the same RRF from script 03. Chunks found by many variants rise to the top.

PROOF - AFTER table in Case C

The top 3 are all from the travel reimbursement policy: the taxi rule AND how to submit the claim.

Ask: What can go wrong when an LLM rewrites the question? *(It can change the meaning or add details. Log the rewrite; it also costs one extra LLM call per question.)*

05 Everything in Qdrant

Scripts 01-04 kept chunks in Python lists so students could see every step. Script 05 puts the same chunks into the real vector database and reads them back. Collection name: **advanced_retrieval** (your policies_issue and policies_fix collections are not touched).

What you are showing

A vector database stores **points**. A point has three parts: an **id**, a **vector** (the meaning, 384 numbers here) and a **payload** (a JSON dict: the chunk text plus every metadata field). Search uses the vector; filtering and showing the source use the payload.

This is where scripts 01 and 02 pay off. The table-aware chunks from 01 become points whose text carries the headers. The metadata from 02 becomes payload fields, so Qdrant can filter on region and year while it searches, exactly like matches() did by hand.

Before you run it

```
docker start qdrant
python 01_table_aware_chunking.py --no-pause # writes output/chunks_table_aware.json
python 05_store_in_qdrant.py
```

Step 1 says "Connected to Qdrant in Docker at http://localhost:6333". If Docker is off it says so and uses a local folder.

Step	What is on screen	What to say
1	Which Qdrant it connected to, and the dashboard link	Same Qdrant as the main project, separate collection.
2	25 chunks: 15 from the .txt policies + 10 from the handbook (5 table_row, 1 table, 4 text)	These are the outputs of scripts 01 and 02.
3	Collection created: size 384, distance COSINE	Every vector must have exactly 384 numbers. COSINE = the similarity we computed by hand.
4	ONE point printed as JSON: the L3 row, id 17	Point = id + vector + payload. Show the payload fields.
5	Every stored point: id, source, type, region, year, first 3 vector numbers, text	This is what is really in the database.
6	Leave question searched in Qdrant without and with a filter	Script 02's filter, now done by the database.
7	Table 1 stored as 5 points, then a vector search inside those rows	The table from script 01 lives in the database row by row.

Code: building the payload (Step 2)

05_store_in_qdrant.py lines 65-74

```
65  # a) the 6 policy documents: text + metadata from their --- header (same as script 02)
66  for c in load_knowledge_base():
67      chunks.append({"text": c["text"], **c["metadata"], "chunk_type": "text"})
68
69  # b) the handbook chunks that script 01 produced (table rows, whole table, sentence text)
70  handbook_meta = {"title": "Employee Handbook", "department": "HR", "region": "India",
71                   "year": 2025, "doc_type": "handbook", "source": "employee_handbook.pdf"}
72  for c in json.loads(TABLE_CHUNKS_FILE.read_text()):
73      chunks.append({"text": c["text"], **handbook_meta,
74                     "chunk_type": c["metadata"]["type"], "page": c["metadata"]["page"]})
```

Line	What it does - say this
67	Policy chunks: ** spreads the metadata dict, so the payload is FLAT: text, region, year, ... (Qdrant filters on top-level keys).
70-71	The handbook PDF has no --- header, so we give its metadata here: HR, India, 2025.
73-74	Handbook chunks from script 01's JSON file, keeping chunk_type (table_row / table / text) and page.

Code: collection and points (Steps 3-4)

05_store_in_qdrant.py lines 86-91

```
86  if qdrant.collection_exists(COLLECTION):
87      qdrant.delete_collection(COLLECTION)            # start clean every run
88  qdrant.create_collection(
89      COLLECTION,
90      vectors_config=models.VectorParams(size=VECTOR_SIZE, distance=models.Distance.COSINE),
91  )
```

Line	What it does - say this
86-87	Delete the old collection first, so every run starts clean.
88-90	Create the collection: vectors of size 384 compared with COSINE distance.

05_store_in_qdrant.py lines 103-119

```
103 points = []
104 for i, (chunk, vector) in enumerate(zip(chunks, vectors)):
105     points.append(models.PointStruct(
106         id=i,                           # a simple number is enough here
107         vector=vector.tolist(),         # 384 numbers = the MEANING of the text
108         payload=chunk,                  # text + every metadata field = what we FILTER on and SHOW
109     ))
110
111 example = next(i for i, c in enumerate(chunks) if "Grade: L3" in c["text"])
112 console.print("[bold]One point, exactly as it is sent to Qdrant:[/bold]")
113 console.print(json.dumps({
114     "id": points[example].id,
115     "vector": f"[{', '.join(f'{x:.3f}' for x in vectors[example][:6])}, ... 384 numbers]",
116     "payload": points[example].payload,
117 }, indent=2))
118
119 qdrant.upsert(COLLECTION, points=points)            # upsert = insert, or overwrite if the id exists
```

Line	What it does - say this
105-108	**PointStruct** = one row of the database: id (a number), vector (384 floats), payload (the chunk dict).
111-117	Print the L3 row point exactly as it is sent, so students see all three parts.
119	upsert = insert, or overwrite if the id already exists. All 25 points in one call.

Code: reading back and searching (Steps 5-7)

05_store_in_qdrant.py lines 126-126

```
126 stored, _ = qdrant.scroll(COLLECTION, limit=100, with_payload=True, with_vectors=True)
```

Line	What it does - say this
126	scroll() reads points straight from the database, with payload and vectors. No search, just 'show me what is stored'.

05_store_in_qdrant.py lines 140-147

```
140 def qdrant_search(query, search_filter=None, limit=4):
141     hits = qdrant.query_points(
142         COLLECTION,
143         query=embed([query])[0].tolist(),   # the question as a vector
144         query_filter=search_filter,         # None = search every point
145         limit=limit,
146     ).points
147     return hits
```

Line	What it does - say this
141	query_points() = vector search inside Qdrant.
143	The question becomes a vector with the same model as the chunks.
144	query_filter: None searches every point; a Filter limits the search to matching payloads.

05_store_in_qdrant.py lines 176-181

```
176 user_filter = models.Filter(must=[
177     models.FieldCondition(key="region", match=models.MatchAny(any=["India", "Global"])),
178     models.FieldCondition(key="year", match=models.MatchValue(value=2025)),
179 ])
180 console.print("\nfilter = region in [India, Global]  AND  year == 2025")
181 show_hits("AFTER - with Qdrant filter", qdrant_search(QUERY, user_filter))
```

Line	What it does - say this
176	Filter(must=[...]) = every condition must be true (AND).
177	MatchAny = the payload value must be one of the list. Same as $in in script 02.
178	MatchValue = must equal 2025. Same as {"year": 2025}.
181	Same question, now with the filter. Only India/Global 2025 points can come back.

05_store_in_qdrant.py lines 188-192

```
188 row_filter = models.Filter(must=[
189     models.FieldCondition(key="source", match=models.MatchValue(value="employee_handbook.pdf")),
190     models.FieldCondition(key="chunk_type", match=models.MatchValue(value="table_row")),
191 ])
192 rows, _ = qdrant.scroll(COLLECTION, scroll_filter=row_filter, limit=20, with_payload=True)
```

Line	What it does - say this
189-190	Filter on source AND chunk_type: only the handbook's table rows.
192	scroll() with a filter: list the 5 points that together store Table 1.

> **PROOF - Step 7 on screen**
>
> Five points, one per grade, each with its headers. The vector search inside those rows should put the L3 row first. If L4 comes first, the script explains why: the rows differ only by 'L3' vs 'L4', which is exactly why script 03 adds keyword search.

Show it in the Qdrant dashboard (Docker only)

1	Open http://localhost:6333/dashboard in the browser.
2	Open Collections and click advanced_retrieval. You also see policies_issue and policies_fix from the main project.
3	Browse the points. Each one shows its payload: text, source, region, year, chunk_type. Find point 17 (the L3 row).
4	Compare with a point from policies_fix to show the same idea in the main project (labels such as status).

Ask: Which part of a point does the search use, and which part does the filter use? *(Search uses the vector. The filter uses the payload.)*

If something goes wrong

Symptom	Fix
No module named 'rag_core'	cd advanced_retrieval first, then run the script.
No module named sentence_transformers / rich / qdrant_client	source ../.venv/bin/activate, then pip install -r requirements.txt
Hangs at "Embedding chunks"	The first run downloads the model (about 90 MB). Do the warm-up command before class.
Gemini call failed: 429	Free-tier quota. The script switches to OFFLINE answers by itself.
Gemini call failed: 404 model	Set a current GEMINI_CHAT_MODEL in production_rag/.env.
05: chunks_table_aware.json not found	Run python 01_table_aware_chunking.py --no-pause first.
05: 'not reachable (is Docker running?)'	docker start qdrant, then run again. Or continue with the local folder (no dashboard).
05: local storage 'already in use'	Another script has advanced_retrieval/qdrant_local open. Close it and run again.
Script does not pause	Pausing only works in an interactive terminal. Use the VS Code terminal, not the Run button.
Tables wrap badly	Widen the terminal, or: COLUMNS=120 python 03_hybrid_retrieval.py

One-line summary for the end of class

Table chunking	Bad chunks can't be rescued later: keep headers with every row.
Metadata filtering	Similarity measures topic, not validity: filter before you rank.
Hybrid search	Keyword and vector fail differently: run both, merge with RRF.
Query rewriting	Clean up the question before you search with it.
Qdrant	A point = id + vector (for search) + payload (for filters and sources).